

Incident — §11.4.84 working-tree-quiescence violation: subagent in-flight deletions leaked into an unrelated docs commit

Revision: 1 **Last modified:** 2026-07-10T21:30:00Z **Severity:**

Governance incident (§11.4.84 violation) — recovered forward, no data lost. **Authority:** §11.4.84 (working-tree quiescence for subagent commits), §11.4.20/§11.4.70 (subagents never git — conductor reviews+commits), §11.4.113 (absolute no-force-push — recovery MUST be forward), §11.4.6 (no-guessing — root cause proven, not assumed), §11.4.4 (STOP-on-discovery).

1. What happened (FACT)

Commit 9d21dd6a — message docs: CONTINUATION Rev20 + READINESS Rev4 — wave-5 progress (0/Q/DASH), intended to be **docs-only** (I passed `commit_all.sh --paths` listing only `docs/CONTINUATION.* + docs/research/.../READINESS.*)` — **also contained 3 clients/ota-manager source-file DELETIONS (332 deletions):**

```
clients/ota-manager/src/features/layout/app-layout.tsx | 156 ----
clients/ota-manager/src/routes/dashboard.tsx           |  71 ----
clients/ota-manager/src/routes/index.tsx               | 105 ----
```

These were the §11.4.124 zero-importer removals from the **in-flight Stream C-impl** (ota-manager router wiring), whose *other* 27 files (the router rewiring that makes those deletions correct) were still uncommitted in the working tree. So 9d21dd6a shipped a **broken intermediate state**: files deleted, but the rewiring that replaces them absent from HEAD.

2. Root cause (PROVEN, §11.4.6 — not “likely”)

Two conditions combined:

1. **commit_all.sh --paths commits the whole index, not just the paths.** The wrapper (scripts/commit_all.sh:346-350) does `git add -- $COMMIT_EXPLICIT_PATHS` on a clean index, then (:404) a **bare git commit -m ...** with NO pathspec. A bare commit snapshots the ENTIRE staged index — so anything *already staged* by another actor is swept in alongside the intended paths.
2. **Stream C-impl staged its deletions with git rm.** C-impl’s report claims “no git writes,” but the 3 files were staged as deletions in the index (only `git rm` does that; plain `rm` leaves them unstaged-and-tracked). A subagent running `git rm` is itself a violation of §11.4.20/§11.4.70 (subagents never git — conductor reviews+commits).

Sequence: C-impl `git rm`d the 3 dead files → they sat staged in the shared index → my next `commit_all.sh --paths` docs commit’s bare `git commit` swept them in.

This is precisely the §11.4.84 forensic failure mode (the JWT-verify mutation-residue case study in the anchor itself): a concurrent actor’s staged change riding into an unrelated commit through a shared checkout.

3. Impact

- 9d21dd6a (pushed to all 4 mirrors) is a broken intermediate: ota-manager at that commit references deleted files. Bounded blast radius — a transient tip state, and no release/tag points at it (§11.4.185 gates release anyway).
- No data lost; no secret leaked; no mutation marker landed (scanned).

4. Recovery (FORWARD ONLY — §11.4.113 no-force-push)

9d21dd6a is immutable (force-push STRICTLY FORBIDDEN). Recovered by **completing** the work forward: independently verified C-impl’s full delta (tsc --noEmit exit 0, vitest 36/36, no new untracked source, mutation-scan clean, dist/ artifact excluded) and committed the 27

completing source files as 512df0eb, restoring a coherent tsc-0 tip.
Verified post-commit: 27 clients files in 512df0eb, 0 dist/ files, tsc 0 at the tip, all 4 mirrors 0/0 FF.

5. Prevention (tracked)

1. **Item E elevated from nice-to-have to important.** `commit_all.sh`
--paths MUST commit ONLY the named paths — either `git commit -- $COMMIT_EXPLICIT_PATHS` (pathspec-limited commit) or reset the index to HEAD before staging so the bare commit can only contain the explicitly-added paths. This wrapper bug turns any pre-staged change into a §11.4.84 leak. (Careful change to the commit tool itself — its own §11.4.115 test: pre-stage a decoy file, run --paths on an unrelated path, assert the decoy is NOT in the resulting commit.)
2. **Reinforce §11.4.20/§11.4.70 in subagent dispatch prompts:**
subagents MUST NOT run ANY git command — not `git add`, not `git rm`, not `git commit`. For §11.4.124 removals a subagent uses plain `rm` (leaves the deletion unstaged) and reports it; the conductor stages+reviews+commits. My dispatch prompts said “do NOT run any git command” but did not call out `git rm` for removals specifically — tighten the wording.
3. **Conductor pre-commit index hygiene:** always `git diff --cached --name-only` and confirm it matches the intended --paths set BEFORE committing (I now do an index-clean precheck before every --paths commit; this incident predates that habit).

Sources verified

Sources verified 2026-07-10: - `git commit/add/rm` index semantics — <https://git-scm.com/docs/git-commit> (pathspec form) - Constitution §11.4.84 (working-tree quiescence) — the governing anchor.